On cherche à créer puis à résoudre algorithmiquement des labyrinthes.

Un peu de vocabulaire

Dans un premier temps, pour identifier plus clairement les étapes de construction et de résolution, nous introduirons le contexte de travail et du vocabulaire le concernant.

Visuellement, un labyrinthe correspond à un rectangle dans lequel des cases sont creusées (en blanc) ou non. La figure 1 en montre un exemple.



FIGURE 1 – Un labyrinthe représenté dans un rectangle de taille 5×7 cases.

Dans l'ensemble du TP, on fera les hypothèses suivantes :

- les dimensions du rectangle sont impaires ;
- les « bords » du labyrinthes sont nécessairement des murs, c'est-à-dire non creusées. Dans les définitions suivantes, nous supposerons que les cases concernées ne sont pas sur les bords du labyrinthe ;
- nous appellerons **pièces** les cases d'indices tous les deux impairs ; dans un labyrinthe terminé, toutes les pièces doivent être creusées ;
- nous appellerons **pilliers** les cases d'indices tous les deux pairs ; dans un labyrinthe terminé, aucun pillier ne doit être creusé ;
- nous appellerons **cloisons** les cases dont exactement un indice est impair ; la forme du labyrinthe obtenu dépend des cloisons qui sont creusées et celles qui ne le sont pas ;
- un labyrinthe sera dit **bien construit** s'il existe toujours un chemin entre deux pièces empruntant des cloisons creusées et d'autres pièces, et que de plus il n'existe pas de cycle, ce qui garantira qu'un tel chemin est unique.

Un labyrinthe peut s'implémenter de manière abstraite par un graphe, dont les sommets sont les pièces et dont les arêtes sont les cloisons creusées. Pour la suite, on choisit une numérotation ligne par ligne. Ainsi, le graphe G_0 correspondant au labyrinthe représenté figure 1 peut se schématiser par la figure 2.

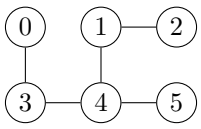


FIGURE 2 – Le graphe G_0.

On remarque alors qu'un labyrinthe bien construit est un arbre couvrant du graphe contenant toutes les cloisons comme arêtes (qu'on appellera graphe grille de taille $\ell \times c$).

1 Éléments de programmation

En Ocaml, on implémente les graphes par un triplet formé d'un tableau de listes d'adjacence, ainsi que du nombre de lignes et du nombre de colonnes **du graphe** (pas du rectangle d'affichage), de telle sorte que si `(g, l, c)` représente un labyrinthe, alors la taille du tableau `g` est $\ell \times c$.

```
type laby = int list array * int * int
```

On trouvera sur le dépôt le fichier `affichages.ml` à télécharger. Ce fichier contient la définition des fonctions suivantes :

- `afficher_laby : laby -> int -> unit` affiche un labyrinthe dans une fenêtre graphique, la largeur en pixels de chaque case étant donnée par le deuxième argument ;
- `dessiner_laby : laby -> (int * int) list -> int -> float -> unit` prend en argument un labyrinthe, une liste d'arêtes du graphe, un entier désignant une largeur en pixels et un flottant, et dessine le labyrinthe dans l'ordre des arêtes fournies, chaque case de la largeur indiquée, et un délai, en secondes, entre chaque arête donné par le flottant. Attention, tout délai inférieur à $0,001$ seconde sera considéré comme nul par la fonction ;
- `dessiner_resolution : laby -> int array -> int list -> int -> float -> unit` dont l'utilisation sera rendue plus claire après le palier 1 ;
- des fonctions de manipulation d'une file de priorité min, impérative (donc modifiable), dont les priorités sont des entiers et dont les valeurs sont de type quelconque. Les noms étant explicites, on infèrera leur utilisation par leur signature. La file de priorité autorise la présence de doublons (valeurs avec la même priorité).
  * `creer_vide : unit -> 'a fileprio` ;
  * `est_vide : 'a fileprio -> bool` ;
  * `ajouter : 'a fileprio -> int -> 'a -> unit` ;
  * `extraire_min : 'a fileprio -> int * 'a` ;
- une fonction utilitaire `melange : 'a list -> 'a list` qui prend en argument une liste et renvoie la même liste mélangée aléatoirement.

Pour tout le TP, vous devrez écrire vos fonctions dans un nouveau fichier que celui fourni. Dans un interpréteur (`utop` ou `ocaml`), on tapera (une seule fois, sauf si on redémarre l'interpréteur) la commande `#use "affichages.ml";;` pour faire l'interprétation du fichier fourni.

1. Créer un objet de type `laby` correspondant au labyrinthe donné par le graphe G_0.
2. Afficher le labyrinthe créé à la question précédente.
3. Dessiner le labyrinthe de la question 1, dans l'ordre d'un parcours en largeur depuis le sommet 0 (on créera la liste des arêtes à la main).

On remarque que dans un graphe de labyrinthe, un sommet est toujours de degré au plus 4.

4. Écrire une fonction `coords (s: int) (l: int) (c: int) : int * int` qui prend en argument un numéro de sommet et le nombre de lignes et colonnes d'un graphe grille et renvoie un couple formé du numéro de ligne et du numéro de colonne du sommet s (en commençant à zéro).
5. Écrire une fonction `voisins_grille (s: int) (l: int) (c: int) : int list` qui prend en argument un numéro de sommet et le nombre de lignes et colonnes d'un graphe grille et renvoie la liste des voisins de s dans ce graphe. La liste renvoyée sera de taille 2 si s est dans un coin, de taille 3 si s est un bord, et de taille 4 sinon.

2 Algorithme de Prim

Il n'existe pas de meilleure façon de générer un labyrinthe, juste des algorithmes qui créent des structures différentes. Pour l'ensemble de la génération de labyrinthes, nous utiliserons de l'aléatoire pour les différencier. Afin que chaque nouvelle interprétation donne des résultats différents, on pourra utiliser une graine aléatoire en tapant, au début du fichier, la commande : `let _ = Random.self_init ()`. Enfin, on rappelle que `Random.int n` renvoie un entier choisi aléatoirement et uniformément entre 0 et $n-1$.

L'algorithme de Prim est un algorithme de recherche d'arbre couvrant minimal dans un graphe connexe. Sa présentation dans le cas général est la suivante :

Entrée : Graphe $G = (S, A, f)$ pondéré non orienté connexe
Début algorithme
  Poser $B = \emptyset$.
  Poser $R = \{s_0\}$ un sommet choisi arbitrairement.
  Poser $\mathcal{F} = \{(s_0, t) \mid t \text{ voisin de } s_0\}$ une file de priorité selon les poids des arêtes.
  Tant que $R \neq S$ **Faire**
    Retirer de $\mathcal{F}$ l'arête (s, t) de poids minimal.
    Si $t \notin R$ **Alors**
      Ajouter t à R.
      Ajouter $\{s, t\}$ à B.
      Pour u voisin de t **Faire**
        Ajouter (t, u) à $\mathcal{F}$.
  Renvoyer (S, B)

Pour l'appliquer à la construction de labyrinthe, il suffit d'attribuer à chaque arête un poids aléatoire. Plutôt que de générer tous les poids d'arêtes initialement, on les génère à la volée, chaque fois qu'une arête est ajoutée à la file de priorité.

6. Écrire une fonction `laby_prim (l: int) (c: int) : laby * (int * int) list` qui prend en argument des entiers ℓ et c et renvoie un couple formé de :
   - un labyrinthe obtenu par l'application de l'algorithme de Prim à un graphe grille de taille $\ell \times c$, avec des poids d'arêtes aléatoires, pas nécessairement tous distincts, choisis entre 0 et $\ell \times c - 1$. On partira du sommet d'indice 0 ;
   - la liste des arêtes du graphe correspondant au labyrinthe, dans l'ordre où elles ont été ajoutées à B dans l'algorithme de Prim.
7. Tester la fonction `laby_prim` en affichant et en dessinant le labyrinthe obtenu. On pourra jouer sur la taille des cases en pixels et sur le délai d'affichage (qu'on pourra mettre à zéro pour de très grands labyrinthes).

Palier 1

3 Résolution de labyrinthe

On considère que la résolution d'un labyrinthe de taille $n = \ell \times c$ consiste à trouver le chemin dans le graphe du sommet 0 au sommet $n - 1$.

Pour éviter de « piéger » les méthodes de résolutions, lorsqu'il y a plusieurs choix possibles pour continuer un parcours, on les explorera dans un ordre aléatoire. On pourra utiliser à cet effet la fonction `melange` qui est fournie.

3.1 Résolution en profondeur

On propose dans un premier d'effectuer un parcours en profondeur pour résoudre le labyrinthe. On choisira d'arrêter le parcours dès que le sommet final $n - 1$ a été atteint.

8. Écrire une fonction `reso_prof ((g, l, c): laby) : int array * int list` qui prend en argument un labyrinthe et renvoie un couple formé de :
   - un tableau de prédécesseurs `pred` tel que pour $s \in S$, `pred.(s)` vaut :
     * -1 si $s = 0$
     * -2 si s n'a pas été atteint au cours de l'exploration ;

    * le parent de $s \neq 0$ dans l'arbre G enraciné en 0 sinon ;
  - la liste des sommets atteints dans l'ordre du parcours en profondeur préfixe.
9. En utilisant la fonction `dessiner_resolution`, tester la fonction précédente. Les arguments de `dessiner_resolution` seront, dans l'ordre :
   - le labyrinthe ;
   - le tableau de prédécesseurs `pred` ;
   - la liste des sommets dans l'ordre où ils ont été atteints ;
   - la taille en pixels d'une case ;
   - le délai de latence en secondes entre le dessin de deux arêtes.

3.2 Algorithme A*

L'algorithme A* permet d'accélérer la résolution d'un labyrinthe dans de nombreux cas. Pour ce faire, on suppose que toutes les pondérations sont égales à 1 et on utilise comme heuristique h pour un sommet s sa distance Manhattan au sommet d'arrivée, c'est-à-dire la norme 1 de la différence (la somme des valeurs absolues des différences de coordonnées (ligne, colonne)).

À la différence de l'algorithme A* classique, on ne cherche pas ici un plus court chemin (de toute façon ce chemin est unique), mais une manière d'obtenir ce chemin en explorant le moins de sommets possibles. Ainsi, la priorité donnée aux sommets dans la file de priorité correspondra uniquement à la valeur de l'heuristique, et pas à la somme coût + heuristique.

10. Écrire une fonction `astar ((g, l, c): laby) : int array * int list` qui résout un labyrinthe en utilisant l'algorithme A*. Les valeurs de retours seront sous le même format que pour la fonction `reso_prof`.
11. Tester la fonction en dessinant la résolution.

Palier 2

4 Autres méthodes de génération

Toutes les fonctions de génération de labyrinthe auront pour signature `int -> int -> laby * (int * int) list` et seront construites avec des arguments et valeurs de retour sous le même format que la fonction `laby_prim`.

4.1 Génération en profondeur

Le principe de la génération en profondeur consiste à effectuer un parcours en profondeur du graphe grille, en parcourant les listes d'adjacence dans un ordre aléatoire, et à conserver l'arborescence ainsi obtenue.

12. Écrire une fonction `laby_prof` qui génère un labyrinthe en profondeur.

4.2 Traversée aléatoire

Pour cette génération, nous allons devoir utiliser une structure de données permettant de retirer un élément aléatoire de la structure. Comme pour une structure de pile ou de file, nous souhaitons des complexités constantes pour les différentes opérations.

On utilisera le type suivant :

```
type 'a randdata = {data : 'a array; mutable taille : int}
```

où `data` représente un tableau contenant les données à stocker, dont la taille correspond au nombre maximal d'éléments présents dans la structure à un instant donné et `taille` correspond au nombre d'éléments effectivement présents dans la structure. Comme pour les files implémentées par tableau, les éléments dont les indices sont supérieurs à `taille` pourront être arbitraires.

13. Écrire une fonction `push : 'a -> 'a randdata -> unit` qui ajoute un élément à la structure.
14. Écrire une fonction `pop : 'a randdata -> 'a` qui prend en argument la structure, retire un élément aléatoire et le renvoie.

Le principe de la traversée aléatoire ressemble à celui de la génération en profondeur, sauf qu'on utilise une table aléatoire au lieu d'une pile dans le parcours.

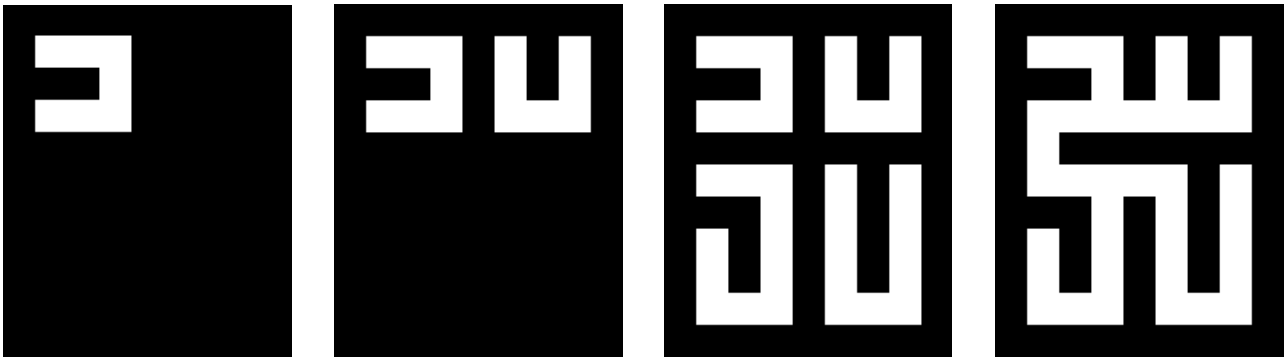
15. Écrire une fonction `laby_alea` qui génère un labyrinthe selon la traversée aléatoire.

4.3 Génération récursive

Cette méthode permet de créer des labyrinthes rectangulaires de la manière suivante :

- on découpe la grille en 4 zones, de tailles équilibrées au maximum ;
- dans 3 des quatre murs de séparation (choisis aléatoirement), on creuse un trou dans une section (choisie aléatoirement) ;
- on recommence récursivement pour chacune des quatre zones, sauf si ce sont des pixels ou des rectangles dont l'une des dimensions vaut 1.

Voici par exemple des étapes de création d'un labyrinthe de taille 9×11 :



16. Écrire une fonction `laby_rec` qui génère un labyrinthe récursivement.

Palier 3

4.4 Algorithme de Wilson

L'algorithme de Wilson est particulier, car au lieu de partir du labyrinthe déjà creusé, il part d'une pièce non reliée, et crée un chemin aléatoirement pour relier cette pièce au reste du labyrinthe. Le principe est le suivant :

- on marque le sommet 0 comme creusé ;
- pour chaque sommet s non vu du graphe (parcourus dans un ordre aléatoire) :
  - on lance un parcours en profondeur (dite procédure de sondage) depuis s dans le graphe grille et on distingue :
    - si on tombe sur un sommet t déjà creusé, on marque comme creusés tous les sommets du chemin trouvé de s à t, et on conserve toutes les arêtes de ce chemin, dans l'ordre de s à t ;
    - si on tombe sur un sommet t déjà vu par le parcours en profondeur depuis s, on revient en arrière (on supprime tout le cycle de l'exploration), et on repart de t ;
    - sinon, on continue le parcours en profondeur en parcourant les voisins dans un ordre aléatoire.

L'algorithme peut être frustrant à ses débuts, car tant qu'il y a peu de pièces dans le labyrinthe, la procédure de sondage peut durer longtemps jusqu'à rencontrer une pièce déjà creusée.

17. Écrire une fonction `wilson` qui génère un labyrinthe selon ce principe.

5 Autres méthodes de résolution

5.1 Parcours en largeur

La résolution de labyrinthe par un parcours en largeur fonctionne de la même manière que le parcours en profondeur, à l'exception près qu'on utilise une structure de file au lieu d'une structure de pile.

18. Écrire une fonction `reso_larg` qui effectue cette résolution.

    On rappelle que les files sont implémentées dans le module `Queue`*, à l'aide des fonctions* `Queue.create`*,* `Queue.push`*,* `Queue.pop` *et* `Queue.is_empty`*.*